AUTONOMOUS UNIVERSITY OF THE STATE OF MEXICO

UAEM VALLE DE MÉXICO UNIVERSITY CENTRE

Modeling an Incremental Learning Scheme with Distributed Memory in Artificial Neural Networks

RESEARCH PROTOCOL

P r e s e n t e d B y

Eng. Luis Ángel González Hernández

Director: Dr. Víctor Manuel Landassuri Moreno
Co-Director: Dr. Leandro L. Minku
Tutor: Dr. Maricela Quintana Lopez

Atizapán de Zaragoza, State of Mexico. November 2025

Centro Universitario
UAEM Valle de México

Contents

List of Figures

List of Tables

Modeling an Incremental Learning Scheme with Distributed Memory in Artificial Nerural Networks

April 23, 2026

Abstract

Incremental learning represents a fundamental challenge in the field of artificial neural networks, mainly due to the phenomenon of catastrophic forgetting, which leads to the loss of previously acquired knowledge when new information is introduced. Although several strategies have been proposed to address this challenge—including regularisation-based methods, replay mechanisms, and parameter isolation—most suffer from limitations in scalability, dependence on stored data, or excessive growth in model complexity, leaving room for more efficient and biologically grounded alternatives. This work proposes a biologically inspired model based on the extension of the approach by [3], whose dual-weight architecture, while effective in mitigating forgetting through short- and long-term memory interaction, has not been explored beyond two memory layers and shows progressive knowledge loss when the number of incremental training stages increases significantly. The proposed extension introduces multiple duplicated weight layers through the progressive duplication of synaptic weights as a mechanism for distributed memory consolidation. The research will be conducted using the OptDigits dataset under an incremental six-block scheme, evaluating different configurations with two, four, and six duplicated memory layers. It is expected that the extended model will achieve higher knowledge retention ($RM \geq 90\%$) and a lower forgetting rate ($F \leq 10\%$) compared to the original dual-weight model, while maintaining a stable computational cost. The anticipated results aim to contribute to a deeper understanding of the stability–plasticity balance in neural networks and to open new possibilities for the development of more efficient and sustainable continual learning systems.

Keywords: Incremental learning; Catastrophic forgetting; Artificial neural networks; Distributed memory; Continual learning.

1 Introduction

Artificial Intelligence (AI) is a field of knowledge that aims to develop machines capable of emulating human behaviour and reasoning, allowing interaction that is almost indistinguishable from that with another human being. This area not only focuses on the creation of intelligent systems but also on developing tools that facilitate and optimise everyday activities.

A subfield of AI is Machine Learning (ML), which studies algorithms capable of autonomously learning tasks. Within this field, Artificial Neural Networks (ANNs) are among the most recognised techniques, as they use mathematical processes to solve complex tasks. ANNs have proven to be useful in areas such as predicting 5G network capacity based on daily traffic [47], as well as in the classification of materials such as metals and rocks using fuzzy logic [36].

ANNs are composed of artificial neurons that simulate biological ones. In this context, the chemical processes that occur in the brain are computationally mimicked through signals that travel between artificial neurons, which will hereafter be referred to as "neurons". This distributed and parallel structure grants ANNs a remarkable learning capacity [22].

In the context of machine learning, when a technique such as ANNs focuses on learning a specific task from a fixed dataset, it is considered a *single-task algorithm without memory*, as the model is trained once and does not

update its knowledge over time. Since their inception, this has been one of the most widely used approaches in ANNs [8]. However, real-world scenarios often require models to continuously incorporate new information—whether in the form of new samples, new classes, or entirely new tasks—without retraining the entire model from scratch. This necessity gives rise to the concept of Incremental Learning, which seeks to update the model's knowledge progressively as new information becomes available, while preserving what has already been learned.

A concept derived from incremental learning is that of memory in AI, which explores how a learning algorithm may forget previously acquired information when incorporating new data. This issue, analogous to human memory loss, poses challenges for both machines and humans. Closely related to this, the field of Continual Learning (CL) has emerged as a prominent research direction, extending incremental learning to deep learning architectures and focusing on the ability of models to learn from a continuous stream of data without forgetting previously acquired knowledge [5, 26]. Despite significant progress, current approaches in this field—including regularisation-based methods such as EWC [15], replay-based strategies such as iCaRL [30], and parameter isolation techniques such as Progressive Neural Networks [35]—still face limitations in scalability, dependence on stored data, or unbounded growth in model complexity, making them unsuitable for resource-constrained scenarios [5].

In this context, Multilayer Perceptrons (MLPs) remain a relevant architecture, particularly in the emerging field of TinyML, where machine learning models must operate on devices with severely limited memory and computational resources. Deep learning approaches are often too heavy for such scenarios, making lightweight architectures such as MLPs a practical and actively explored alternative [8]. Investigating incremental learning in MLPs is therefore not only theoretically meaningful, but also practically motivated by real-world deployment constraints.

In response to these challenges, various approaches have been designed to mitigate catastrophic forgetting. One example is the work by [3], which proposes the use of ANNs with dual weights, where the first set of weights behaves as short-term memory and the second as long-term memory, emulating the hippocampus–neocortex interaction observed in biological systems. The experiments conducted by [3] demonstrate an improvement in incremental learning tasks, reducing information loss compared with previous implementations such as the Learn++ algorithm [18, 6]. However, a key limitation of this approach is that, as the number of incremental training stages grows, the knowledge acquired from earlier datasets is progressively lost, reducing its effectiveness in prolonged learning scenarios. Furthermore, the dual-weight configuration has not been explored beyond two memory layers, leaving open the question of whether additional intermediate memories could provide a more gradual and stable consolidation process. This research aims to address this gap directly, extending Bullinaria's model towards a multi-level memory architecture and evaluating its impact on knowledge retention, forgetting rate, and computational efficiency.

This research aims to explore new configurations of duplicated weights, extending and expanding the results obtained in [3].

2 Problem Statement

Artificial Neural Networks (ANNs), particularly Multilayer Perceptrons (MLP), possess the ability to learn tasks and solve prediction and classification problems. By using learning algorithms such as Backpropagation, it is possible to adjust the weights of an ANN so that it can address a specific task. MLPs are a common form of ANN, composed of multiple layers of neurons, where the input layer receives the data and the output layer provides the predictions. The intermediate layers, also known as hidden layers, perform non-linear transformations that allow the network to learn complex representations of the data. However, many real-world tasks generate additional information over time. An example of this can be seen in financial time series behaviour or weather forecasting in a specific region. In this context, incremental learning emerges as a relatively unexplored approach that allows the model to learn new information without the need to retrain the entire model with both old and new data.

In current machine learning models, when a dataset is used to train a specific model, that model is functional only for that dataset and the information it represents. However, when it becomes necessary to incorporate new information into the model, this new data must be collected, added to the existing dataset, and the entire model must then be retrained to integrate the new data. This means that, in practice, retraining requires combining all available data—both old and new—into a single dataset, which can be computationally expensive and impractical when data arrives continuously over time.

In this scenario, if a network trained with an initial dataset d_1 needs to incorporate knowledge from a second dataset d_2, one of two situations arises: either the model is retrained from scratch using the combined dataset

$d_1 \cup d_2$, preserving prior knowledge at a high computational cost, or d_2 is used on its own to update the model, in which case the knowledge acquired from d_1 will be lost in the process due to catastrophic forgetting. If an incremental learning model is not used, both d_1 and d_2 must be combined into a single dataset and the ANN retrained to include the new knowledge (d_2). In contrast, when incremental learning is employed, the ANN is first trained with d_1 and then with d_2, maintaining minimal information loss from d_1. If additional information (d_3) becomes available in the future, the ANN can be trained incrementally with d_3, minimising the loss of data from d_1 and d_2.

This work is based on the research conducted by [3], which uses a duplicated-weight configuration in ANNs. In this configuration, the network's weights are duplicated and assigned different learning rates to the layers. The first set of weights, associated with a high learning rate, simulates short-term memory by quicklyThe first set of weights, associated with a high learning rate, simulates short-term memory by quickly learning new tasks. This approach, proposed by [3], aims to enhance the incremental learning capability of ANNs by introducing weight duplication. In this approach, each connection in the network has two versions of its weights: one simulating short-term memory and the other representing long-term memory.

Short-term memory employs a high learning rate, allowing rapid adaptation to new information, but it is also more prone to forgetting previously learned knowledge. Conversely, long-term memory is trained with a lower learning rate, learning more slowly but retaining previously acquired information more effectively.

During training, both sets of weights are updated in parallel according to their respective learning rates. When making predictions, the outputs are derived from a combination of both weight sets. This combination mechanism bears conceptual similarities to ensemble learning, where multiple models contribute to a final prediction [16, 9]. However, unlike traditional ensembles that maintain entirely separate networks, the approach by [3] integrates both weight sets within a single architecture, using their interaction to balance short-term adaptation and long-term knowledge retention. It is worth noting that ensemble methods have also been explored in the context of incremental and data stream learning [16, 9], representing a complementary line of research that will be further discussed in the state of the art.

The main issue identified in the incremental learning approach by [3] is that as new datasets are incorporated, the knowledge acquired from the earlier datasets is progressively lost, which becomes less effective if ten or twenty incremental training stages are carried out with new data. This limitation is not exclusive to [3]: more broadly, existing approaches to mitigate catastrophic forgetting still struggle to maintain knowledge retention over long sequences of incremental updates without incurring high computational costs, requiring access to previously seen data, or expanding the model architecture indefinitely [5, 15, 30, 35]. Regularisation-based methods such as EWC [15] impose constraints on weight updates but require estimating parameter importance matrices, which becomes increasingly costly over many tasks. Replay-based methods such as iCaRL [30] depend on storing or regenerating past data, raising privacy and scalability concerns. Parameter isolation methods such as Progressive Neural Networks [35] avoid interference between tasks but grow unboundedly with each new stage. Therefore, it is crucial to explore lightweight, memory-efficient alternatives that can sustain knowledge retention across a large number of incremental stages without these drawbacks.

Therefore, it is crucial to explore new ANN configurations that enhance current methods to reduce the amount of forgotten information as new data arrive. As in previous research, this work is based on the concepts of short- and long-term memory. However, instead of merely duplicating the weights and using two learning rates, the idea proposed here is to create multiple copies of the weights, each with a distinct learning rate.

3 State of the Art

3.1 Search Strategy

To prepare the present state of the art, the PRISMA 2020 methodology (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) was adopted, which provides a standardised framework for conducting systematic reviews of scientific literature in a transparent, rigorous, and reproducible manner.

The aim was to identify and analyse relevant research related to incremental learning, continual learning, and strategies to mitigate *catastrophic forgetting* in artificial neural networks.

The literature search covered the period from 2010 to 2025, in order to encompass both foundational works—such as those proposed by Bullinaria and other authors in the 2010s—and the most recent advances in the field. This period was selected on the grounds that, from 2010 onwards, scientific interest in retention and forgetting mechanisms within machine learning became consolidated.

The search process was carried out between 2010 and 2025 in the following academic databases:

- IEEE Xplore

- ScienceDirect
- SpringerLink
- Scopus
- Google Scholar
- Advancing Computing as a Science & Profession
- arXiv (Cornell University)

Boolean combinations of keywords in English and Spanish were employed, including: *"incremental learning"*, *"lifelong learning"*, *"continual learning"*, *"catastrophic forgetting"*, *"artificial neural networks"* and *"artificial intelligence"*. The terms were adapted to the search options of each database in order to maximise coverage and retrieve the most relevant and up-to-date studies. The main search queries used were the following:

- `("incremental learning" OR "continual learning" OR "lifelong learning") AND "artificial neural networks"`
- `("catastrophic forgetting" OR "knowledge retention") AND ("neural networks" OR "deep learning")`
- `("incremental learning" OR "continual learning") AND ("stability-plasticity" OR "catastrophic forgetting") AND ("regularisation" OR "replay" OR "parameter isolation")`
- `("aprendizaje incremental" OR "aprendizaje continuo") AND ("redes neuronales artificiales" OR "olvido catastrófico")`
- `("continual learning" OR "incremental learning") AND ("TinyML" OR "resource-constrained" OR "embedded systems")`
- `("dual weights" OR "synaptic duplication" OR "distributed memory") AND ("incremental learning" OR "neural networks")`
- `("ensemble learning" OR "data stream") AND ("incremental learning" OR "concept drift")`

These queries were adapted syntactically to the search interface of each database, preserving their semantic structure. Searches were conducted both with and without truncation operators where supported, and results were filtered by publication date (2010–2025) to ensure relevance and recency.

3.2 Inclusion and Exclusion Criteria

With the purpose of ensuring the relevance and methodological quality of the selected studies, the following criteria were established:

Inclusion criteria:

- Publications between 2010 and 2025.
- Articles that explicitly address incremental or continual learning in artificial neural networks.
- Research that analyses the phenomenon of *catastrophic forgetting* or proposes associated solutions.
- Studies with applications in areas such as robotics, computer vision, cybersecurity, or adaptive systems, as well as works that evaluate incremental or continual learning methods on benchmark datasets without focusing on a specific application domain. This includes articles that use standard datasets (e.g., MNIST, CIFAR, or similar benchmarks) primarily for algorithmic evaluation and comparison purposes, provided that the main contribution addresses incremental learning, catastrophic forgetting, or knowledge retention in artificial neural networks. Articles were not excluded on the basis of their application domain alone, but rather on whether their methodological contribution was relevant to the research questions addressed in this work.

Exclusion criteria:

- Review articles without empirical evidence.
- Works without full-text access or with preliminary results.
- Duplicated publications or those with evident methodological bias.

3.3 Selection Process

The initial search yielded a total of 1,246 records across the seven databases consulted. After removing 312 duplicates, 934 unique articles were retained. During the screening phase (title and abstract review), 886 studies were excluded for not meeting the inclusion criteria. The most frequent reasons for exclusion were: (i) focus on deep learning architectures such as CNNs, RNNs, or Transformers, outside the MLP-based scope of this work; (ii) continual learning applied to large language models or vision-language models [23, 38]; (iii) replay-based or parameter isolation methods dependent on stored data or architectural expansion; or (iv) absence of empirical evaluation on lightweight benchmark datasets. The remaining 48 articles were assessed in full text, and, ultimately, 18 met the methodological and thematic standards required for qualitative synthesis. It is worth noting that the relatively small number of retained articles reflects the strict application of the inclusion and exclusion criteria defined in Section 3.2, rather than a scarcity of relevant work in the field. A broader overview of the continual learning landscape can be found in [5, 26, 23].

The selection process is summarised in Table 1 and in the PRISMA flow diagram shown in Figure 1.

Table 1: Summary of the article selection process according to PRISMA 2020.

Stage	**Number of studies (*n*)**
Records identified in databases	1,246
Duplicates removed	312
Records after duplicate removal	934
Records excluded after title/abstract screening	886
Articles assessed in full text	48
Articles excluded for lack of applicable results	30
Studies included in qualitative synthesis	18

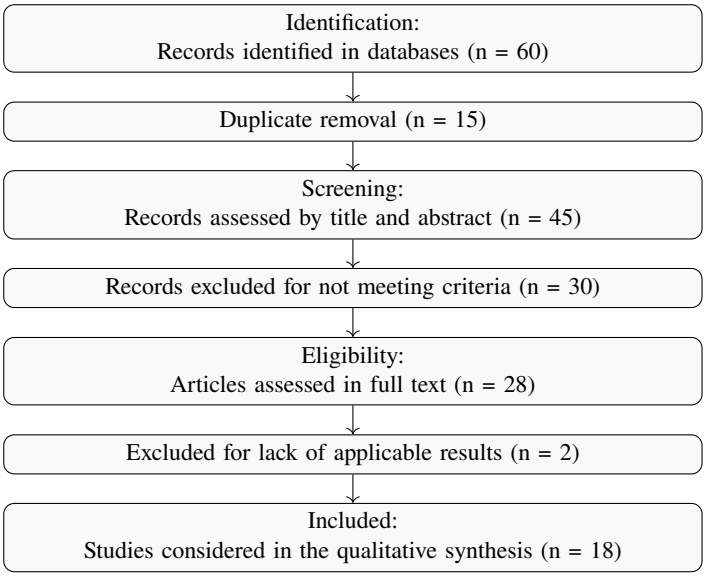

Figure 1: PRISMA flow diagram for study selection.

3.3.1 Synthesis of Results

The 18 studies analysed offer a comprehensive overview of incremental learning in artificial neural networks, addressing different domains, architectures, and strategies to mitigate catastrophic forgetting. The comparative analysis made it possible to identify six main thematic strands, reflecting both conceptual evolution and the most relevant practical applications in the area:

- **Robotics and adaptive control:** In this field, incremental learning is applied mainly to the control of intelligent prostheses, exoskeletons, and autonomous robots, where systems must continuously adapt to changing environmental or user conditions. Lippi et al. [21] highlight that incremental models allow dynamic recalibration without the need for global retraining, enabling more precise and energy-efficient

responses. These findings support the idea of maintaining dual-memory mechanisms, where one part of the system consolidates stable experiences whilst another adjusts to new inputs.

- **Computer vision and pattern recognition:** In this line, there is extensive use of convolutional neural networks (*CNNs*) with continual learning mechanisms applied to image classification, semantic segmentation, and autonomous driving. Wei et al. [44] show that combining *replay* strategies with selective regularisation improves the retention of visual features. However, such methods often rely on storing previous samples, which implies a higher computational and memory cost. In this context, biologically inspired models such as [3] offer a lighter alternative by not requiring explicit memory.

- **Cybersecurity and traffic analysis:** Cerasuolo et al. [4] implement the *MEMENTO* model for incremental classification of encrypted traffic, demonstrating the applicability of continual learning in anomaly detection scenarios. Their approach combines weight consolidation strategies with temporal stability metrics, reinforcing the relevance of hierarchical memory management in systems that must respond in real time.

- **General methods and theoretical approaches:** Among the most influential advances are elastic regularisation methods proposed by Kirkpatrick et al. [15], episodic memory mechanisms such as *iCaRL* [30], and architectures with progressive neuronal expansion described by [3]. These proposals reflect different approaches to the stability–plasticity dilemma: whilst EWC and MAS prioritise the preservation of knowledge, progressive models promote structural flexibility and sustained adaptation.

- **Emerging trends (2024–2025):** Recent research such as that of Hacohen et al. [11] evidences *LIFO* (Last In, First Out) patterns in continual learning, where examples learned more quickly tend to be better retained, whilst more difficult ones are forgotten more easily. For their part, Sarkar et al. [37] propose *EVCL+* (Elastic Variational Continual Learning Plus), a methodology that combines Fisher regularisation with variational learning to protect the most significant parameters. These trends reflect growing interest in adaptive optimisation of weight space, aligning with Bullinaria's philosophy of adjusting weights via controlled duplication rather than resorting to external memories.

- **Recent taxonomies and comparisons:** De Lange et al. [5] establish a structured classification of continual learning methods into three major families: Replay, Regularisation-based, and Parameter Isolation. Their results show that models such as *PackNet* and *iCaRL* achieve greater knowledge retention, while *Memory Aware Synapses (MAS)* exhibits greater robustness to hyperparameter variation. Nonetheless, most of these approaches require explicit memory management or architectural expansion, which increases complexity. Therefore, it is advisable to explore alternatives based on internal weight consolidation, such as those proposed by Bullinaria, which balance efficiency and stability with minimal computational cost.

3.3.2 Model Comparison and Recent Trends

Below is a concise comparison of the main approaches identified in the literature, organised into two blocks: classic models (2009–2019), which laid the foundations of incremental learning, and recent models (2020–2025), which reflect current trends in stability, plasticity, and memory consolidation.

Table 2: Concise comparison of classic and recent approaches in incremental learning.

Author (Year)	Model / Technique	Family	Main contribution	Limitation
Classic Models (2009–2019)				
Bullinaria (2009) [3]	Dual Weights	Internal consolidation	Mitigates forgetting without *replay*.	Not scalable (>2 memories).
Kirkpatrick et al. (2017) [15]	EWC	Regularisation	Preserves critical weights (Fisher).	High computational cost.
Zenke et al. (2017) [46]	Synaptic Intelligence	Regularisation	Efficient on-line update.	Sensitive to hyperparameters.
Li & Hoiem (2016) [19]	LwF	Distillation	Avoids storing past data.	Depends on old outputs.
Rusu et al. (2016) [35]	Progressive NN	Isolation	Avoids task interference.	Grows indefinitely.
Rebuffi et al. (2017) [30]	iCaRL	Replay + Distillation	High per-class retention.	Requires exemplar memory.
De Lange et al. (2021) [5]	Survey	Taxonomy	Classifies methods (Replay, Reg., Isolation).	Does not propose a model.
Recent Models (2020–2025)				
Wei et al. (2020) [44]	Multi-Domain Adaptation	Replay + Reg.	Improves visual retention.	Requires partial memory.
Lillicrap et al. (2020) [20]	Backpropagation and the Brain	Biological insp.	Relates biological and ANN learning.	Theoretical, not directly incremental.
Lippi et al. (2021) [21]	Gait Phase ID	Robotic application	Incremental in adaptive control.	Task-specific.
Cerasuolo et al. (2024) [4]	MEMENTO	Weight consolidation	Incremental without *replay*; temporal stability.	Limited domain.
Hacohen & Tuytelaars (2023) [11]	Forgetting Order	Forgetting dynamics	Analyses LIFO forgetting pattern.	Preliminary results.
Sarkar (2025) [37]	EVCL+	Variational reg.	Adaptive stability.	Implementation complexity.
Liu et al. (2025) [23]	CL for VLMs	Survey	Modern taxonomy (large models).	Focus on LLMs/VLMs.

3.3.3 Conclusion of the State of the Art

The analysis of the state of the art reveals significant progress in the development of incremental learning models applied to various domains, from robotics and computer vision to cybersecurity and natural language processing. This progress has enabled artificial intelligence systems to gradually adapt to new environments without requiring complete retraining, demonstrating that the ability to learn continuously is essential to achieving truly intelligent and autonomous behaviour.

Nevertheless, despite the achievements, the reviewed literature shows that incremental learning still faces fundamental challenges related to stability, plasticity, and computational efficiency. In particular, current models tend to fall into one of the two extremes of the stability–plasticity dilemma: either they retain too much prior knowledge, hindering the incorporation of new information, or they adapt rapidly to recent data at the cost of forgetting what has already been learned. This balance remains one of the main barriers to achieving genuinely continual learning.

Contemporary strategies to mitigate *catastrophic forgetting*—such as regularisation methods, *replay*, and parameter isolation—have shown partial effectiveness but present limitations that affect their scalability and practical applicability. Regularisation methods (such as EWC [15] or MAS [2]) require estimates of parameter importance, which implies high computational cost; *replay*-based methods [30, 33] depend on storing or generating old data, affecting privacy and performance; and parameter isolation methods (such as *PackNet* [25] or *Progressive Networks* [35]) tend to increase network size indefinitely, compromising efficiency in prolonged learning contexts. These limitations are particularly critical in the context of TinyML [43], where devices operate under severe constraints in terms of computational power, memory, and storage capacity. In such scenarios, methods that depend on importance matrix estimation, data buffers, or unbounded architectural growth become impractical or entirely infeasible, reinforcing the need for lightweight, parameter-efficient alternatives that can consolidate knowledge internally without relying on external memory or structural expansion.

These limitations underscore the need to explore alternatives that integrate the benefits of such strategies without increasing model complexity or relying on explicit memories. In this context, biologically inspired approaches gain relevance, as they offer mechanisms that emulate the brain's memory consolidation—particularly the interaction between the hippocampus (short-term memory) and the neocortex (long-term memory)—thus allowing gradual and efficient knowledge retention.

Within this line, the model proposed by [3] is a key contribution, introducing a dual-memory system based on the duplication of synaptic weights. This design allows one part of the network to process recent information dynamically (short-term memory), while another part consolidates stable knowledge (long-term memory). Such an approach offers a viable alternative to architectures that depend on external memories or structural expansion, providing a simpler and more efficient solution with solid neurocomputational underpinnings.

However, the reviewed studies also indicate that Bullinaria's model, although conceptually robust, has been little explored and lacks extensions that analyse its behaviour with different memory configurations or variations in weight duplication. Most investigations have been limited to replicating the original implementation without assessing its potential to optimise the stability–plasticity trade-off in more complex or prolonged incremental learning scenarios.

Therefore, the present research is formulated as a direct extension of that work, with the aim of evaluating and expanding the dual-memory model towards an architecture with multiple weight duplications, analysing its impact on knowledge retention, forgetting rate, and training efficiency. Through this proposal, the objective is to determine whether increasing the number of intermediate memories can favour a more gradual transition between acquisition and consolidation, thereby reducing catastrophic forgetting without significantly increasing computational cost.

In summary, whilst modern strategies focus on architectural complexity or intensive use of resources, the proposed approach returns to a fundamental principle: biologically inspired simplicity as the basis for learning stability. If proven effective, this model could represent an intermediate step between traditional approaches and truly adaptive systems, providing a more sustainable, interpretable, and efficient alternative within the field of incremental learning in artificial neural networks.

4 Problem Formulation

This work addresses the **class-incremental learning** problem [5, 40], where a neural network must learn to recognise new classes over time while retaining knowledge of previously learned ones, without access to past training data during subsequent learning stages.

Formal Definition

Let $\mathcal{D} = \{D_1, D_2, \ldots, D_T\}$ be a sequence of T training blocks, where each block $D_i = \{(\mathbf{x}_j, y_j)\}_{j=1}^{n_i}$ consists of n_i labelled examples. Each input $\mathbf{x}_j \in \mathbb{R}^d$ is a feature vector of dimension d, and $y_j \in \mathcal{Y}_i$ is the corresponding class label, where $\mathcal{Y}_i$ denotes the set of classes introduced in block D_i. The class sets are disjoint across blocks, i.e.:

$$\mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset, \quad \forall\, i \neq j$$

such that new classes are introduced at each learning stage. The cumulative set of classes after training on block D_i is defined as:

$$\mathcal{Y}_{1:i} = \bigcup_{k=1}^{i} \mathcal{Y}_k$$

Learning is performed **chunk-by-chunk**: at each stage i, the model receives the entire block D_i and updates its parameters accordingly. Crucially, the model does **not** have access to data from previous blocks $D_1, \ldots, D_{i-1}$ during training on D_i, which is the defining constraint of the incremental learning setting [5, 30].

Learning Objective

At each stage i, the model is parameterised by a set of weight matrices $\mathbf{\Theta}_i = \{\mathbf{W}_1^{(i)}, \mathbf{W}_2^{(i)}, \ldots, \mathbf{W}_n^{(i)}\}$, where n denotes the number of duplicated weight sets. The learning objective at stage i is to minimise the cross-entropy loss over the current block:

$$\mathcal{L}_i(\mathbf{\Theta}) = -\frac{1}{n_i} \sum_{j=1}^{n_i} \sum_{c \in \mathcal{Y}_{1:i}} \mathbf{1}[y_j = c] \log \hat{p}(c \mid \mathbf{x}_j; \mathbf{\Theta})$$

where $\hat{p}(c \mid \mathbf{x}_j; \mathbf{\Theta})$ is the predicted probability for class c given input $\mathbf{x}_j$, and $\mathbf{1}[\cdot]$ is the indicator function.

The Stability–Plasticity Trade-off

The central challenge in this setting is to simultaneously satisfy two competing objectives [5, 26]:

- **Plasticity:** the model must adapt its parameters to correctly classify examples from the new block D_i, i.e., minimise $\mathcal{L}_i$.
- **Stability:** the model must retain its ability to correctly classify examples from all previous blocks $D_1, \ldots, D_{i-1}$, i.e., avoid *catastrophic forgetting* [10, 15].

Formally, after training on block D_i, the ideal model should minimise the following global objective:

$$\mathcal{L}_{\text{global}}(\mathbf{\Theta}) = \sum_{k=1}^{i} \mathcal{L}_k(\mathbf{\Theta})$$

However, since data from past blocks is not available, directly minimising $\mathcal{L}_{\text{global}}$ is infeasible. This is the fundamental constraint that motivates the development of incremental learning strategies.

Proposed Approach

This work proposes to address this trade-off through the **progressive duplication of synaptic weight sets**, inspired by the dual-weight architecture of [3]. Given n duplicated weight sets $\{\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_n\}$, each set $\mathbf{W}_k$ is associated with a distinct learning rate η_k, such that:

$$\eta_1 > \eta_2 > \cdots > \eta_n$$

where $\mathbf{W}_1$ acts as *short-term memory* (high plasticity) and $\mathbf{W}_n$ acts as *long-term memory* (high stability). The final prediction is derived from a combination of all weight sets:

$$\hat{p}(c \mid \mathbf{x}; \mathbf{\Theta}) = f\left(\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_n, \mathbf{x}\right)$$

where $f(\cdot)$ denotes the combination function, whose specific form (simple average, weighted average, or learned combination) will be investigated as part of the experimental design (see Section **??**).

5 Objectives

5.1 General Objective

To design and implement an artificial neural network for incremental learning inspired by the principle of short- and long-term memory, using more than two layers with duplicated weights for digit recognition. The model aims to reduce the information loss associated with catastrophic forgetting, improving the retention of previously acquired knowledge in comparison with prior works reported in the literature.

5.2 Specific Objectives

1. **Reproduce** the algorithm proposed by [3] for digit recognition through learning with short- and long-term memory, using the original parameters and configurations described by the author.

2. **Organise** the dataset into training and testing partitions according to the methodology established in [3], and validate the initial results of the baseline model using comparable performance metrics.

3. **Extend** the reference algorithm to incorporate more than two layers of duplicated weights, evaluating its impact on the stability and plasticity of incremental learning using the same dataset.

4. **Compare** both implementations quantitatively (original model and extended model) based on incremental accuracy, forgetting rate, and memory retention, aiming to demonstrate a significant reduction in catastrophic forgetting compared with previous works [5, 15]. The evaluation will be conducted across multiple datasets to assess the generalisation of the proposed approach, including OptDigits, MNIST [17], Fashion-MNIST [45], and the Speech Commands dataset [42]. For image-based datasets, a lightweight pre-trained encoder will be explored as a preprocessing step to extract compact feature representations suitable for MLP input, following the constraints of resource-limited deployment scenarios consistent with the TinyML paradigm [43].

5. **Explore** the use of lightweight pre-trained encoders as a preprocessing step for image-based datasets, evaluating their compatibility with the proposed incremental architecture under resource-constrained conditions consistent with the TinyML paradigm [43].

6 Hypothesis and Evaluation Criteria

6.1 Research Hypothesis

Incremental learning in artificial neural networks faces the challenge of balancing the stability of previously acquired knowledge with the plasticity required to incorporate new information. Within this context, the following research hypothesis is proposed:

> **General Hypothesis:** The incorporation of additional duplicated memories in a neural architecture based on the model of [3] will increase the retention of previously acquired knowledge by at least 10–15% compared with the original model, while maintaining a comparable computational cost and without requiring explicit replay mechanisms.

This hypothesis is based on the premise that the duplication of synaptic weights acts as a distributed consolidation mechanism, analogous to the transfer processes from short- to long-term memory observed in biological systems. Thus, the aim is to demonstrate that the inclusion of multiple intermediate memories can mitigate catastrophic forgetting and promote a smoother transition between incremental learning phases.

6.2 Evaluation Criteria

To validate the proposed hypothesis, the following quantitative and qualitative evaluation criteria are considered, applied to each incremental block of the experiment:

- **Average Incremental Accuracy (IA):** a measure of the overall performance of the model, considering the progressive accumulation of knowledge throughout the training blocks. After training on block B_i, the model is evaluated on all previously seen blocks $B_1, B_2, \ldots, B_i$ using their respective held-out test sets. The average incremental accuracy after block i is computed as:

$$IA_i = \frac{1}{i} \sum_{j=1}^{i} A_{i,j}$$

  where $A_{i,j}$ denotes the accuracy measured on the test set of block B_j after the model has been trained up to block B_i. This formulation follows the evaluation protocol adopted in recent continual learning literature [5, 30].

- **Forgetting Rate (F):** the percentage loss of accuracy on a previously learned block after incorporating new information. Specifically, after training on block B_i, the forgetting rate with respect to block B_j ($j < i$) is calculated as:

$$F_{i,j} = \frac{A_{j,j} - A_{i,j}}{A_{j,j}} \times 100$$

where $A_{j,j}$ is the accuracy on block B_j immediately after training on it, and $A_{i,j}$ is the accuracy on the same block after subsequently training on blocks $B_{j+1}, \ldots, B_i$. The overall forgetting rate F is then averaged across all previous blocks:

$$F_i = \frac{1}{i-1} \sum_{j=1}^{i-1} F_{i,j}$$

This metric is consistent with the backward transfer measure adopted in [5, 15, 24].

- **Memory Retention (RM):** the proportion of knowledge preserved at the end of incremental training, comparing the final performance with the initial one:

$$RM = \frac{A_f}{A_0} \times 100$$

where A_f is the final accuracy and A_0 the initial accuracy.

- **Computational Cost (CC):** the average training time per block and resource usage, aiming to determine the efficiency of the proposed architecture compared with the baseline model.

- **Stability–Plasticity (EP):** qualitative analysis of the balance between the preservation of prior knowledge and the model's adaptability to new tasks.

The results obtained for each metric will be analysed comparatively among the model variants (with different numbers of duplicated memories) and against the reference scheme proposed by [3]. The objective is to determine whether the proposed approach improves knowledge retention without compromising adaptability or significantly increasing computational cost.

7 Justification

Artificial Neural Networks (ANNs) represent one of the most relevant tools within the field of artificial intelligence due to their ability to learn from data and solve complex problems of classification, prediction, and pattern recognition. However, traditional machine learning methods present a critical limitation: when new data blocks are incorporated, the model tends to deteriorate in performance and forget part of the information previously acquired—a phenomenon known as catastrophic forgetting [3].

Despite recent advances in deep architectures and regularisation techniques, neural networks still lack enduring memory mechanisms that allow them to retain information over the long term without degrading their performance. In contrast, human beings have the ability to learn new tasks without significantly erasing previous knowledge, integrating new information progressively. Although this property is associated with the biological structure of the brain—particularly with processes such as synaptic consolidation—from a computational standpoint it is feasible to explore strategies that emulate this behaviour, optimising the storage and updating of knowledge in artificial models.

From a technical perspective, there are no theoretical limitations that prevent experimentation with new neural architectures, loss functions, or training schemes that promote progressive learning. The challenge lies in enabling the network to assimilate new data without overwriting previously acquired knowledge, thereby achieving a balance between the stability and plasticity of the system. Overcoming this challenge would enable the development of more robust and adaptive models capable of operating continuously in dynamic environments such as intelligent monitoring systems, robotics, or real-time applications where data are constantly updated.

In environments where neither incremental nor continual learning is employed, each data incorporation requires retraining the entire model with all the available information. While recent continual learning approaches have partially addressed this by retraining only on a selected subset of the most relevant past data [30, 16], this still implies storing and managing historical data, which raises concerns regarding memory consumption, scalability, and data privacy in resource-constrained or sensitive deployment scenarios [5]. This process is costly in terms of computational time, energy consumption, and hardware requirements. Conversely, an incremental approach allows the model to process only the new data, updating its knowledge without repeating the entire

training process. In addition to optimising resources, this characteristic can provide both economic and technological advantages over other methods, by reducing computational cost, development time, and algorithmic complexity in continuous learning applications.

There are currently various tools that facilitate the construction and training of neural networks. For the experimental development of this research, the Google Colab environment will be primarily employed, as it offers free infrastructure based on Graphics Processing Units (GPU), full integration with the Python programming language, and an interactive environment that supports fast and reproducible experimentation.

Likewise, the model implementation will be carried out using the PyTorch library, which is widely recognised for its flexibility and dynamism in handling computational graphs. PyTorch allows defining, modifying, and debugging network architectures intuitively during execution, thanks to its design based on dynamic tensors and its syntax closely aligned with pure Python. These characteristics facilitate the exploration of different training schemes, the customisation of loss functions, and the integration with scientific libraries such as NumPy and Matplotlib.

In this sense, PyTorch is considered a suitable tool for the development and evaluation of incremental learning models, as it provides a flexible environment that supports both experimentation and performance analysis under progressive data incorporation scenarios. Its use will allow the validation of the effects of incremental learning in mitigating catastrophic forgetting, contributing to the design of more efficient, sustainable, and human-like neural network architectures.

8 Delimitation

In this research, only multilayer perceptron (MLP) neural networks will be used. It should be noted that this is not the only type of neural network, as there are also Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Radial Basis Function Networks (RBFN) [34]. Furthermore, optimisation techniques such as genetic algorithms will not be addressed, as the study is limited exclusively to exploring performance improvements in neural networks with more than two duplicated weight sets within an incremental learning context. The choice of Multilayer Perceptrons (MLPs) as the base architecture is deliberately motivated by the constraints of the TinyML paradigm [43], where machine learning models must operate on devices with severely limited memory and computational resources. In such scenarios, deep learning architectures are often prohibitively expensive, making lightweight models such as MLPs a practical and well-suited alternative. This delimitation therefore reflects not only a methodological decision, but also a practical consideration aligned with real-world deployment requirements in resource-constrained environments. Similarly, the study is restricted to the aforementioned networks, excluding deep neural network models and the use of additional datasets.

9 Consequences

If the experiment is successful, the following outcomes are expected:

1. A reduction in the forgetting of previously learned information.
2. A decrease in training time.

As previously mentioned, it will be possible to incorporate new data without the need to retrain the model with all the existing information.

Classification or prediction tasks will be able to integrate new information into the artificial neural networks without retraining the model on all historical data, thereby reducing the bottleneck associated with the training process.

10 Theoretical Framework

10.1 Optical Digit

The Optical Digit dataset is a classical benchmark in pattern recognition and classification algorithm evaluation, particularly within the context of Optical Character Recognition (OCR). Its original purpose was to provide a simple, reproducible, and well-balanced testbed for comparing supervised learning models without the computational cost associated with high-resolution image datasets. In other words, it is a lightweight yet sufficiently expressive benchmark for studying the generalisation capacity of different approaches.

Each instance in the dataset consists of an 8×8 grayscale image (as shown in Figure 2); each image encodes a digit in the range 0–9. The dataset contains a total of 5,620 images and maintains balanced class distribution —each digit has the same number of samples. This detail is non-trivial: by avoiding class frequency bias, performance metrics (accuracy, F1-score, etc.) more fairly reflect the differences among models and preprocessing pipelines.

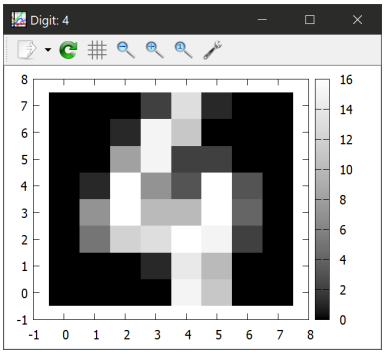

Figure 2: Optical Digit Dataset

Thanks to its balance between simplicity and practicality, the Optical Digit dataset is widely used in various areas of artificial intelligence research, such as:

1. Digit classification [27];
2. Pattern recognition [12];
3. Model comparison and benchmarking [29];
4. Teaching and training in supervised learning [13].

Among its main advantages are: (*i*) very fast training and validation curves (ideal for hyperparameter and architecture tuning), (*ii*) ease of experiment reproducibility (few transformations and minimal preprocessing), and (*iii*) suitability as a case study for incremental learning methodologies, where the effect of introducing sequential data blocks can be examined without excessive computational cost.

10.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the functioning of the human brain, designed to process information in a parallel and distributed manner. Each network is composed of a set of simple processing units called artificial neurons, interconnected through weighted links known as synaptic weights. These weights represent the strength of the connection between neurons and determine how much influence one input exerts over another within the network. In this sense, learning in an ANN essentially consists of the progressive adjustment of these weights, seeking to minimise the error between the generated output and the expected one.

Analogous to neurophysiology, where synaptic connections are strengthened or weakened according to experience, ANNs learn by modifying the numerical weights of their connections through training algorithms. This adaptive capacity gives neural networks the ability to recognise patterns, classify information, make predictions, and approximate complex functions without requiring explicit programming.

The simplest model of a neuron—known as the perceptron is the fundamental unit of ANNs. Introduced by Rosenblatt in 1958, the perceptron aims to emulate the behaviour of a biological neuron by combining its inputs through weighted summation, followed by an activation function that determines its output. Mathematically, this model can solve linearly separable problems, where data can be divided by a linear boundary in the input space.

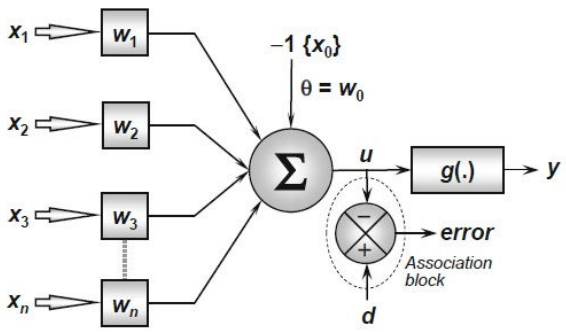

Figure 3: Diagram of a basic Artificial Neural Network.

In Figure 3, the symbol Σ represents the summation operator. Let $x_1, x_2, \ldots, x_n$ be the input variables and $w_1, w_2, \ldots, w_n$ their corresponding synaptic weights. The neuron first computes a weighted linear combination of the inputs and adds a bias term (b) that allows the decision boundary to shift away from the origin. The mathematical model of a neuron can be expressed as:

$$y = \sum_{i=1}^{n} w_i x_i + b,$$

where y represents the output of the neuron. The bias plays an essential role by introducing flexibility into the model, allowing the adjustment of the decision boundary's position without altering the weight values. Thus, a single neuron acts as a linear classifier, separating input data using a hyperplane in the feature space.

However, the perceptron's limitations become apparent when faced with non-linear problems, in which a single boundary is insufficient to correctly divide the classes. This restriction led to the development of architectures with multiple layers of neurons and non-linear activation functions, known as multilayer neural networks (MLPs), capable of modelling more complex relationships between input and output variables.

A single neuron can solve tasks such as AND or OR, but not non-linear problems (e.g., XOR). In such cases, architectures with multiple layers are required, where activation functions introduce the necessary non-linearity to model complex decision surfaces.

Logic Gates and Linear Separability in ANNs. Binary logic gates help illustrate the relationship between linear separability and the representational capacity of an artificial neural network.

- **AND:** outputs 1 only when both inputs are 1. It is linearly separable; a perceptron can implement it, for instance, with $w_1 = w_2 = 1$ and $b = -1.5$. The boundary $x_1 + x_2 - 1.5 = 0$ correctly separates the single positive case $(1, 1)$.

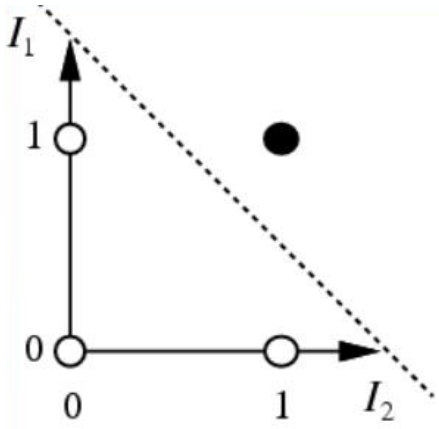

Figure 4: AND logic gate in the input plane.

- **OR:** outputs 1 when at least one input is 1. It is also linearly separable; a perceptron can implement it with $w_1 = w_2 = 1$ and $b = -0.5$. The boundary $x_1 + x_2 - 0.5 = 0$ separates the three positive cases $(1, 0), (0, 1), (1, 1)$ from the negative one $(0, 0)$.

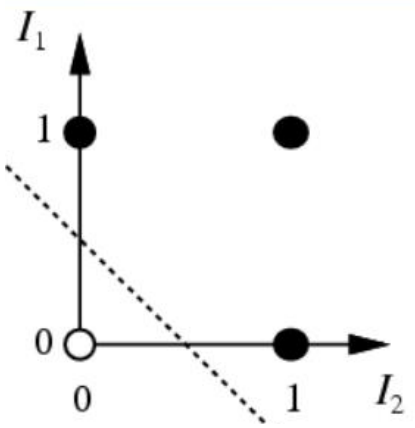

Figure 5: OR logic gate in the input plane.

- **XOR:** outputs 1 only when exactly one input is 1. It is not linearly separable since no single line can simultaneously divide the positive cases $(1, 0), (0, 1)$ from the negatives $(0, 0), (1, 1)$. Solving XOR requires, at minimum, a network with one hidden layer (MLP) that combines multiple linear boundaries into a non-linear decision surface.

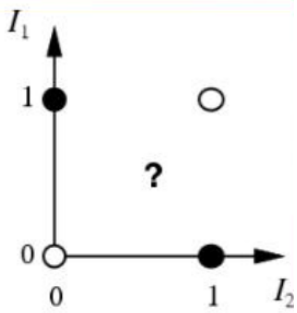

Figure 6: XOR logic gate: example of a non-linearly separable problem.

This relationship is fundamental to the design of neural architectures: when data cannot be separated by a single hyperplane, a multilayer network with suitable activation functions can compose linear boundaries into non-linear forms, thus representing complex problems accurately (see Section 10.3).

10.2.1 Activation Functions

Activation functions are the key component that transform a stacked network of linear transformations into a genuinely non-linear model. Without them, even multiple layers would effectively collapse into a single linear transformation. In problems where the input–output relationship is intrinsically non-linear—as is the case in most real-world phenomena—activation functions enable the network to bend the decision boundary and stabilise the gradient flow during training [31].

Among the most common activation functions are:

- **Step Function:**
  Suitable for simple binary decisions (0/1). Its modern use is mainly pedagogical, as its non-differentiability makes gradient-based optimisation difficult:

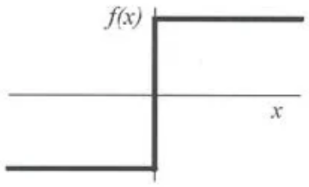

Figure 7: Step Function

$$f(x) = \begin{cases} 0 & : x < 0 \\ 1 & : x \geq 0 \end{cases}$$

- **Sigmoid Function:**
  Produces outputs in the range $[0, 1]$ and allows probabilistic interpretation (e.g., $0.8 \Rightarrow 80\%$). It smooths the decision boundary and facilitates the computation of gradients:

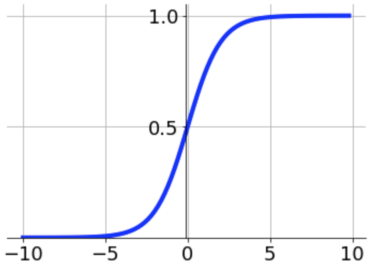

Figure 8: Sigmoid Function

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its parameters are updated through backpropagation (see Section 10.4).

- **Rectified Linear Unit (ReLU):**
  The de facto standard in deep networks due to its efficiency and ability to alleviate the vanishing gradient problem. It activates positive values proportionally and nullifies negatives, thereby accelerating convergence:
  - Vanishing gradient: phenomenon where gradients become very small and early layers stop updating effectively.

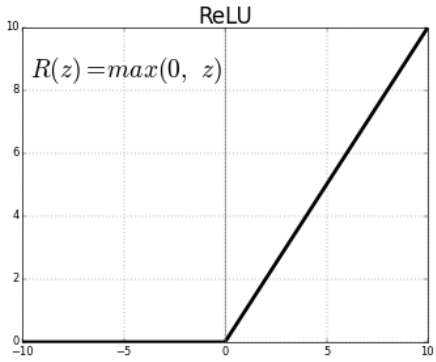

Figure 9: ReLU Function

$$f(x) = \begin{cases} 0 & : x < 0 \\ x & : x \geq 0 \end{cases}$$

- **Softmax Function:**
  Commonly used in the output layer for multi-class classification, transforming activations into probabilities that sum to one:

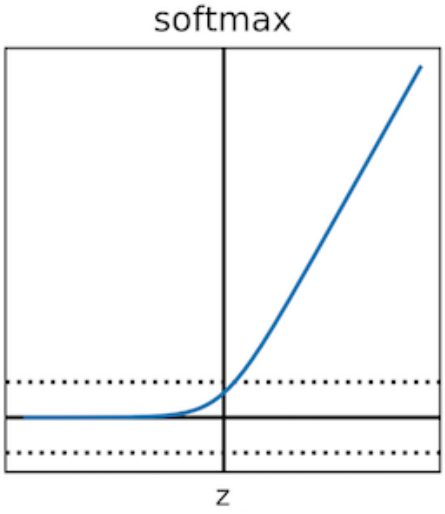

Figure 10: Softmax Function

$$f(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}.$$

- **Hyperbolic Tangent Function:**
  Similar to the sigmoid but with an output range of $[-1, 1]$, useful for centring data around zero; in very deep networks, it may still suffer from the vanishing gradient issue:

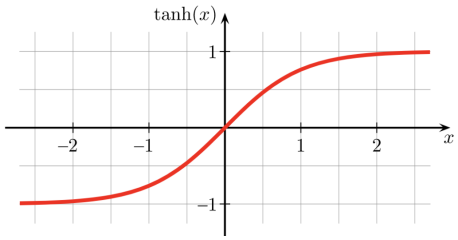

Figure 11: Hyperbolic Tangent Function

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}.$$

In summary, activation functions enable ANNs to handle non-linear relationships, learn complex input–output mappings, and build models more robust to noise. Among the most widely used architectures are multilayer perceptrons and convolutional neural networks, described in the following sections.

10.3 Multilayer Perceptron Neural Networks

Multilayer Perceptron Neural Networks (MLPs) consist of an input layer, one or more hidden layers, and an output layer. The hidden layers contain neurons equipped with activation functions (see Section 10.2.1), responsible for breaking linearity and constructing decision surfaces in higher-dimensional spaces. Thus, problems that a single neuron cannot solve (such as XOR) become tractable by combining multiple layers and activation functions.

Figure 12 illustrates a four-layer MLP: one input layer, two hidden layers, and one output layer. The input layer injects the variables, while non-linear processing occurs in the hidden layers (and, in some designs, also in

the output layer). This modular structure facilitates the addition of depth or width according to the complexity of the problem.

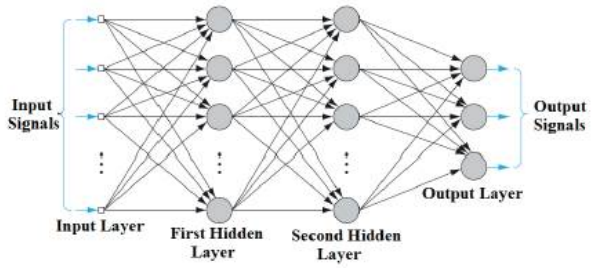

Figure 12: Multilayer Perceptron Neural Network.

Each neuron within a layer receives inputs from all neurons in the previous layer, multiplies them by their corresponding weights, sums the results, and passes them through a non-linear activation function. This process allows the network to approximate any continuous function, as established by the Universal Approximation Theorem, which states that an MLP with at least one hidden layer and sufficient neurons can approximate any function to an arbitrary degree of accuracy.

During training, the MLP learns by adjusting the synaptic weights and biases to minimise a loss function that quantifies the error between the predicted and target outputs. This optimisation process is typically carried out using the backpropagation algorithm (see Section 10.4), which propagates the error backwards through the network to update the parameters layer by layer.

In summary, the MLP architecture provides a powerful and flexible foundation for non-linear modelling, enabling the representation of complex patterns and relationships in data. Its generality and adaptability have made it one of the most widely used neural models in areas such as pattern recognition, time series prediction, and classification tasks.

10.4 Backpropagation Algorithm

The backpropagation algorithm enables the adjustment of the network's parameters (weights and biases) based on the gradient of an error or loss function. It was a key development that extended the capability of the simple perceptron to deep networks capable of solving non-linear problems.

Operationally, the network first performs a forward propagation (feed-forward) to compute the output and the associated error. Then, the error is propagated backwards (backpropagation) through the network by applying the chain rule to compute the partial derivatives of the loss function with respect to each parameter. These derivatives are then used to update the weights and biases via an optimisation algorithm (typically gradient descent or one of its variants). The process requires differentiable activation functions and an appropriately defined loss function [32].

The learning flow can be summarised in two main phases:

1. **Feed-forward:** propagation of the inputs through the network to produce outputs and compute the corresponding error.

2. **Backpropagation:** backward propagation of the error and parameter updates, layer by layer.

During the feed-forward phase, the inputs are multiplied by their respective weights, biases are added, and the results are passed through activation functions to generate outputs. In the backpropagation phase, the error between the predicted and expected outputs is calculated using a loss function, commonly the Mean Squared Error (MSE) for regression or Cross-Entropy for classification. The gradients of this error with respect to each weight are computed recursively from the output layer to the input layer, allowing efficient parameter updates.

Formally, for each weight w_{ij} connecting neuron i in layer $l-1$ to neuron j in layer l, the update rule is expressed as:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial E}{\partial w_{ij}},$$

where E denotes the loss function and η is the learning rate, a hyperparameter that controls the magnitude of weight updates. A small η leads to slower convergence but greater stability, while a large η may accelerate learning but risk overshooting minima.

Modern training schemes often employ adaptive optimisation algorithms such as Adam, RMSProp, or Adagrad, which adjust the learning rate dynamically for each parameter, enhancing convergence in high-dimensional spaces.

A detailed mathematical description of the feed-forward and backpropagation phases can be found in Appendix A (Sections A.1 to A.2).

In summary, backpropagation provides the foundation for training multilayer networks, enabling the automatic adjustment of parameters to minimise error iteratively. Its efficiency and generality have made it a cornerstone of modern deep learning.

10.5 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are deep learning architectures specifically designed for data with spatial or local structure, such as images, video sequences, or visual patterns. Their success lies in their ability to automatically extract hierarchical features through a combination of three complementary types of layers:

- **Convolutional layers:** apply local filters (kernels) to detect low-level features such as edges, textures, and shapes. These layers enable partial translational invariance, meaning that detected features remain recognisable even if they appear in different positions within the input.
- **Pooling layers:** (also known as subsampling or downsampling) reduce the dimensionality of feature maps and computational cost, while preserving the most relevant information. Common techniques include Max Pooling and Average Pooling, which summarise the activation of a region by its maximum or mean value.
- **Fully connected layers:** integrate the high-level features extracted by previous layers to perform final decision-making or classification.

Figure 13 conceptually illustrates the typical structure of a CNN, where an input image passes through successive convolutional and pooling stages before reaching fully connected layers that output the class probabilities.

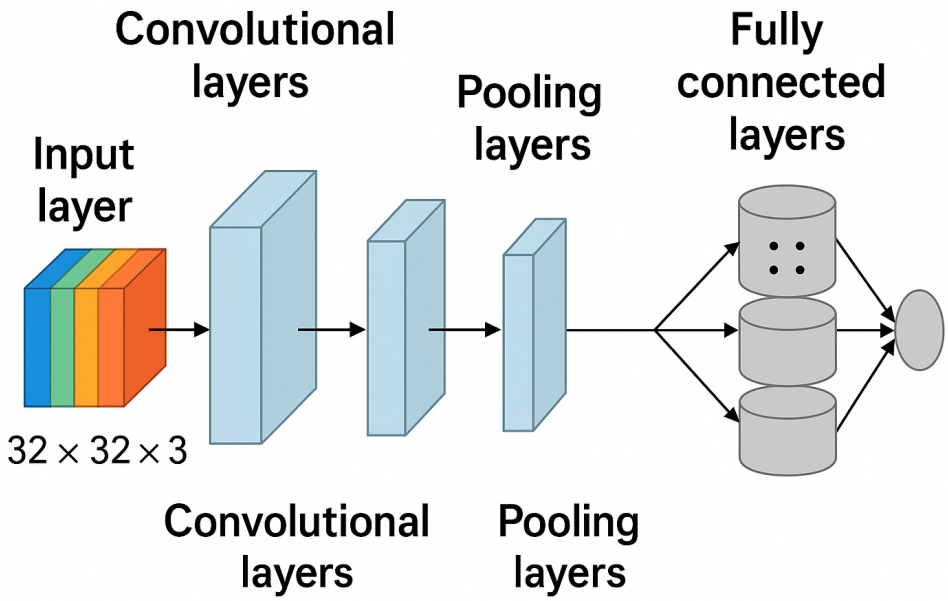

Figure 13: General structure of a Convolutional Neural Network.

The main advantage of CNNs over traditional Multilayer Perceptrons (MLPs) lies in their ability to exploit spatial correlations between neighbouring pixels through local receptive fields and shared weights. This design drastically reduces the number of parameters to be trained, improving computational efficiency and generalisation capability.

CNNs are widely used in various fields of artificial intelligence, including:

- **Computer vision:** object detection, image classification, and scene segmentation.
- **Medical imaging:** automated diagnosis and detection of anomalies in radiographs or MRI scans.
- **Robotics and autonomous systems:** real-time recognition of obstacles, navigation, and environmental understanding.
- **Speech and audio recognition:** feature extraction from spectrograms for voice or music processing.

From an architectural perspective, CNNs can be combined with other models such as Recurrent Neural Networks (RNNs) or Transformers to process spatio-temporal data, enabling applications in video analysis and natural language processing. Their modular and scalable design has made CNNs one of the foundational paradigms of modern deep learning.

10.6 Incremental Learning

The continuous growth of data availability and the increasing demand for systems capable of learning over time have driven the development of Incremental Learning. In this paradigm, training examples are presented sequentially, and the model must adapt to new information without forgetting the knowledge previously acquired. Unlike traditional supervised learning approaches—where the entire dataset is processed simultaneously—in incremental learning, the model is trained progressively, integrating new patterns or tasks over time [8].

This feature is essential in dynamic or non-stationary environments, where data distributions evolve and retraining the model from scratch after every update becomes impractical. For instance, in robotics, systems must continuously adapt to changes in lighting, textures, or human interaction conditions without discarding previously learned information. Similarly, in fields such as cybersecurity, image processing, speech recognition, or recommendation systems, the continuous flow of new data demands models capable of real-time adaptation without degrading their past performance.

Incremental learning draws inspiration from human cognition: people can incorporate new knowledge throughout life while retaining essential information from past experiences. In neural networks, replicating this capability involves balancing two fundamental properties — plasticity, which allows adaptation to new data, and stability, which preserves previous knowledge. This balance, known as the stability–plasticity dilemma, represents one of the core challenges of incremental learning.

However, conventional neural networks suffer from a phenomenon known as catastrophic forgetting, where the integration of new tasks or data leads to the loss of previously acquired information. This occurs because the optimisation process adjusts the network's weights to fit new data, overwriting those that encoded prior knowledge. As a result, the model may perform well on the most recent tasks while exhibiting poor performance on earlier ones [22].

To address this issue, several strategies have been proposed in the literature, grouped into three main categories:

- **Regularisation-based methods:** constrain changes in parameters deemed important for previous tasks, thereby protecting critical knowledge. Representative examples include Elastic Weight Consolidation (EWC) and Synaptic Intelligence (SI).
- **Rehearsal or replay methods:** store a small subset of representative samples or generate synthetic data to periodically retrain the network on past experiences while learning new ones.
- **Parameter-isolation methods:** allocate specific subsets of neurons or layers to individual tasks, thus reducing interference among them.

The overarching goal of these strategies is to enable models to learn continuously without requiring full retraining or constant access to historical data. Consequently, incremental learning has become a central paradigm within Continual Learning, fostering the development of more efficient, adaptive, and autonomous intelligent systems. Ultimately, this approach brings artificial neural networks closer to human-like cognition, where knowledge is accumulated and consolidated progressively over time.

10.6.1 Incremental Learning Algorithms

In general terms, an incremental learning algorithm aims to maintain the ability to learn continuously without retraining the entire model from scratch. Such algorithms are designed under the premise that data are not presented as a static set but rather as a sequence of experiences or information blocks that must be processed

as they arrive. The central challenge is to update the model's parameters without compromising the knowledge already acquired — thereby preventing the phenomenon of catastrophic forgetting.

To achieve this, an incremental algorithm must satisfy the following essential criteria [1]:

1. Learn and update its parameters with each new data sample, whether labelled or unlabelled, without requiring global retraining.
2. Retain previously acquired knowledge while integrating new information.
3. Avoid permanent dependence on the original training data (i.e., no need to store all past examples).
4. Detect and create new classes or clusters when the data introduce previously unseen patterns, and merge or split existing clusters as necessary.
5. Maintain a dynamic and adaptive behaviour when facing concept drift or changes in data distribution.

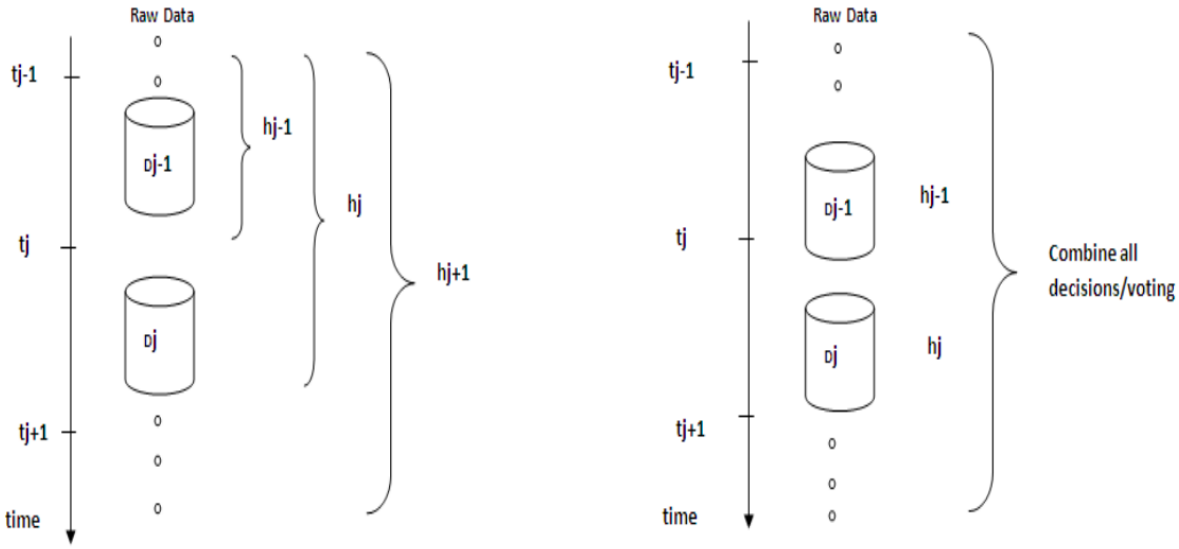

Figure 14: Two traditional approaches to incremental learning.

As illustrated in Figure 14, there are two fundamental historical approaches to incremental learning:

- **Cumulative or sequential approach:** Whenever a new data block D_j arrives, the model discards the previous hypothesis h_{j-1} and trains a new one, h_j, using all available data. Although this strategy often improves global accuracy, it is computationally expensive and does not represent truly continuous learning.
- **Purely incremental approach:** The model receives D_j and directly updates the parameters of the existing hypothesis h_{j-1} based only on the new data. In some cases, ensemble or voting mechanisms are used to stabilise changes between successive iterations.

In both cases, the key practical advantage of incremental learning lies in the fact that the acquired knowledge is stored within the model's parameters (weights, biases, or probabilities), rather than as individual examples. This allows the system to avoid storing large volumes of historical data, achieving scalability and efficiency. This property is particularly relevant in the context of TinyML [43], where devices operate under severe memory and storage constraints, making parameter-based knowledge retention a critical advantage over approaches that depend on maintaining historical data buffers. In such scenarios, the ability to consolidate knowledge directly into the model's weights — without requiring access to past examples — becomes not only desirable but essential for practical deployment.

Formally, given a sequence of training examples $e_1, e_2, \ldots, e_s$, an incremental learning algorithm generates a series of hypotheses $h_0, h_1, \ldots, h_n$ such that each hypothesis h_{i+1} depends only on the previous hypothesis h_i and the current example e_i [8]. This principle enables adaptive and continuous learning, where the model "evolves" through interaction with data over time.

A classical example of an incremental learning system is the COBWEB model [7], which performs hierarchical probabilistic categorisation. COBWEB analyses each incoming instance and decides whether to create a new category, update an existing one, or restructure the hierarchy based on a global utility metric. Thus, the system incrementally builds and refines a knowledge tree without restarting the learning process.

With the advancement of the field, several families of incremental algorithms have been proposed, classified according to their primary strategy:

- **Regularisation-based methods:** Restrict parameter updates for weights considered critical to previous tasks. Examples include Elastic Weight Consolidation (EWC) [15], Synaptic Intelligence (SI) [46], and Memory Aware Synapses (MAS) [2].
- **Replay-based methods:** Use a small buffer of representative samples or generative models to recall past knowledge during training. Examples include iCaRL and Generative Replay techniques.
- **Parameter-isolation methods:** Assign specific subsets of neurons or layers to different tasks, reducing interference between them. Notable examples include Progressive Neural Networks and PackNet.
- **Hybrid methods:** Combine regularisation, replay, and dynamic architectural expansion to balance stability and plasticity in non-stationary contexts.

Within this research line, the work of [3] represents a key reference point. It introduces an alternative biologically inspired approach based on dual-memory architecture: a short-term memory responsible for recent information consolidation, and a long-term memory that preserves stable knowledge. This mechanism is implemented through the duplication of synaptic weights, allowing the model to control forgetting rates and learning speeds independently.

Bullinaria's model employs incremental learning where each data block updates only the short-term network, and information is gradually transferred to the long-term network — effectively preventing direct overwriting of consolidated knowledge. This architecture emulates, in simplified form, the memory consolidation process observed in the human brain (hippocampus–neocortex interaction), providing an efficient solution to mitigate catastrophic forgetting without explicit storage of previous data.

In this study, this principle is adopted and extended to explore whether using more than two duplicated weight sets (multi-level synaptic memory) can further enhance knowledge retention and stability during incremental learning.

10.6.2 Memory Mechanisms in Incremental Learning

The human learning process integrates multiple memory systems that interact in a hierarchical and complementary manner. Inspired by this principle, incremental learning in artificial neural networks seeks to emulate the distinction between **short-term memory** — which stores recent and volatile information — and **long-term memory**, which preserves stable and consolidated knowledge over time.

In computational terms, these mechanisms are represented through different storage structures or by duplicating synaptic weights. Models such as that proposed by [3] implement two distinct sets of weights: one for rapid updating (analogous to the hippocampus) and another for slow consolidation (similar to the neocortex). This architecture allows the network to remain plastic and adaptable during the learning of new patterns, while simultaneously maintaining the stability of previously acquired knowledge.

Figure 15 illustrates the conceptual analogy between biological and artificial memory systems, highlighting how information transitions from short-term to long-term representations.

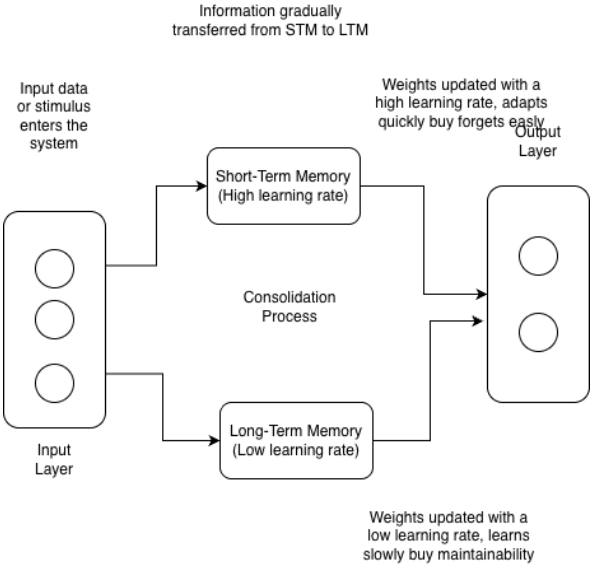

Figure 15: Conceptual analogy between biological and artificial memory systems.

The interaction between these memory components forms the foundation of the present research, which extends the dual-memory approach to a multi-level retention scheme. The objective is to evaluate whether introducing additional layers of synaptic duplication — each with distinct learning rates — enhances knowledge retention and mitigates information loss in incremental learning scenarios.

In this context, the proposed model aims to simulate a more distributed and gradual memory consolidation process, where each memory layer functions as a transition phase between short-term and long-term representations. This progressive mechanism seeks to capture the temporal dynamics of learning, enabling the network to balance adaptability to new information with the preservation of prior knowledge — a property that more closely mirrors the behaviour of biological learning systems.

10.6.3 Strategies to Mitigate Catastrophic Forgetting

One of the greatest challenges in incremental learning is catastrophic forgetting — the phenomenon where a neural network loses part or all of its previously acquired knowledge when learning new tasks or integrating recent data. This issue arises because the optimisation process modifies the network's weights to adapt to new information, thereby overwriting parameters that previously represented prior knowledge. Consequently, the model may achieve high performance on the most recent task while performing poorly on earlier ones.

To address this limitation, the literature has proposed several strategies that aim to balance two fundamental principles: plasticity — the ability to adapt to new data — and stability — the capacity to retain previous knowledge. This balance, known as the stability–plasticity dilemma, remains at the core of research in continual and incremental learning.

Below are the main strategies currently used to counteract catastrophic forgetting:

1. **Regularisation-based methods:**
   These approaches introduce constraints on the parameter update process to prevent drastic changes in weights that were important for previous tasks. A classical example is Elastic Weight Consolidation (EWC), which computes a parameter importance matrix and penalises significant deviations [15]. Modern variants such as Synaptic Intelligence (SI) and Memory Aware Synapses (MAS) follow a similar principle, maintaining a "synaptic memory" of relevant parameters to reinforce their preservation.

2. **Rehearsal or replay-based methods:**
   These techniques retain a small memory buffer of representative past examples or employ generative models to recreate synthetic samples [30, 33]. During the training of new tasks, the network periodically revisits this memory to maintain awareness of prior knowledge. Prominent examples include Incremental Classifier and Representation Learning (iCaRL) [30] and Generative Replay, which uses generative adversarial networks (GANs) or variational autoencoders (VAEs) to simulate previous data distributions [39]. More recent approaches have extended these ideas by combining replay with transformer-based

architectures [41] and by proposing more efficient buffer selection strategies to reduce memory overhead while maintaining knowledge retention [37, 4].

3. **Parameter-isolation or partitioning methods:**
   Instead of reusing the same weights across all tasks, these approaches allocate subsets of neurons or layers exclusively to specific tasks, reducing interference between them. Models such as Progressive Neural Networks and PackNet gradually expand the network's architecture, preserving learned connections while adding new capacities for subsequent tasks. This method is particularly effective when the number of tasks is known or limited.

4. **Knowledge distillation and transfer-based methods:**
   Inspired by Knowledge Distillation, these strategies transfer knowledge from a pre-trained "teacher" network to a new "student" network. During incremental training, the student network learns new tasks while replicating the teacher's predictions for previous ones, thus maintaining previously acquired representations and reducing forgetting.

5. **Hybrid or multi-strategy methods:**
   The most recent approaches combine several of the aforementioned techniques to achieve a more efficient balance between stability and plasticity. These hybrid models integrate regularisation, selective replay, and dynamic architecture expansion, adapting their behaviour according to data characteristics or task complexity [5, 37].

6. **Ensemble-based methods:**
   Ensemble approaches maintain multiple models or weight sets that contribute jointly to predictions, offering a natural mechanism to balance stability and plasticity [16, 9]. Some approaches train a separate model for each batch of incoming data [28], which provides flexibility but introduces challenges related to batch size selection and the management of an ever-growing number of models. Strategies to prune or delete past models are therefore required to keep the ensemble size manageable [16]. Other ensemble methods maintain different models to capture different concepts, sometimes employing concept drift detection to decide when to retire an old model and start training a new one, preserving prior knowledge in the archived model for later retrieval [9]. However, these approaches are not necessarily designed for class incremental learning scenarios, where new classes must be recognised alongside previously learned ones — which is the setting targeted in this work.

   Notably, the dual-weight architecture proposed by [3] can itself be interpreted as a form of ensemble, where two sets of weights — each trained with a different learning rate — contribute jointly to the final prediction. Unlike traditional ensemble methods that maintain entirely separate networks, this approach integrates both weight sets within a single architecture, avoiding the overhead of managing multiple independent models. The present work extends this idea by introducing additional weight sets, further enriching the ensemble-like consolidation mechanism while remaining lightweight and suitable for resource-constrained environments consistent with the TinyML paradigm [43].

Each of these strategies aims, to varying degrees, to emulate the cognitive processes of human learning and memory consolidation. However, most require manual control over model complexity or the partial storage of previous data, which limits their scalability in real-world applications.

In this context, the model proposed by [3] stands out as a biologically inspired alternative that avoids explicit replay mechanisms or external storage structures. Instead, it introduces a dual-memory system through the duplication of synaptic weights, enabling gradual consolidation of new knowledge. This process mirrors the biological transfer of information from the hippocampus to the neocortex, offering a more natural and computationally efficient framework to mitigate catastrophic forgetting.

The present research builds upon and extends this idea, evaluating whether the use of multiple duplicated weight sets — representing several levels of memory consolidation — can offer a more robust and scalable solution to catastrophic forgetting within incremental learning frameworks.

11 Methodology

??

This research is divided into three main stages: the recreation of the base model, the implementation of an extended version of the model, and the comparison of the obtained results. Each of these stages is described in detail below:

1. **Recreation of the base code**
   As a first step, the code presented in [3] will be recreated. It describes the implementation of a multilayer neural network using the backpropagation training algorithm in the Python programming language. This code will serve as the foundation for the following stages.
   Additionally, the Optical Digits dataset will be used and pre-processed to remove invalid records and ensure the quality of the data employed for training and validating the model.

2. **Extension of the base model**
   Subsequently, an extension of the base model will be developed with the aim of experimenting with more complex neural network configurations. Specifically, more than two sets of duplicated weights will be added, which is expected to improve information retention when applying incremental learning.
   To achieve this, experiments will be conducted by gradually increasing the number of duplicated weight sets in order to analyse how this configuration affects the learning capability of the network. In addition, different strategies to combine the outputs of the multiple weight sets will be investigated, as the aggregation mechanism may have a significant impact on the stability–plasticity balance. The combination strategies to be explored include:

   - **Simple average:** the outputs of all weight sets are averaged with equal contribution, following the baseline approach of [3].
   - **Weighted average:** each weight set contributes proportionally to its associated learning rate, assigning greater influence to more stable (slower) memories.
   - **Learned combination:** a trainable scalar parameter is assigned to each weight set, allowing the model to adaptively determine the contribution of each memory level during training, inspired by ensemble weighting strategies [16].
   - **Recency-weighted combination:** more recently trained weight sets receive higher contribution, prioritising plasticity over stability, which may be beneficial in non-stationary data stream scenarios [9].

   The impact of each combination strategy will be evaluated using the metrics defined in Section 6.2, and the most effective configuration will be identified for each incremental learning scenario.

3. **Comparison and analysis of results**
   Once the simulation results are obtained, a comparison will be made between the performance of the base model and that of the extended model. This analysis will include key metrics such as incremental accuracy (IA), forgetting rate (F), memory retention (RM), and computational cost (CC), as defined in Section 6.2.
   The results will allow for evaluating whether the modifications introduced into the neural network architecture significantly improve its performance compared to the original implementation [3].
   Crucially, the proposed model will also be compared against representative existing approaches from each major category of continual learning methods, selected on the basis of their recency, relevance, and availability of open-source implementations compatible with MLP-based architectures. The planned baselines include:

   - **Regularisation-based:** Elastic Weight Consolidation (EWC) [15] and Synaptic Intelligence (SI) [46], both of which have well-established open-source implementations and are directly applicable to MLP architectures.
   - **Replay-based:** Experience Replay (ER) [33] using a fixed-size memory buffer, and the MEMENTO approach [4], which represents one of the most recent replay-based methods with available code and demonstrated applicability in resource-constrained scenarios.
   - **Ensemble-based:** Learn++ [28] as a classical ensemble baseline, and a data stream ensemble variant following the strategies surveyed in [16, 9].
   - **Recent hybrid approaches (2024–2025):** EVCL+ [37], which combines Fisher regularisation with variational learning and represents one of the most recent approaches in the field. Where full implementations are not publicly available, the core strategies described in the original papers will be reimplemented as MLP variants to ensure a fair and reproducible comparison.

   For each baseline, the same incremental training protocol, dataset partitioning, and evaluation metrics will be applied to ensure experimental fairness. Statistical analysis will be conducted to determine whether observed differences in performance are significant. This comparative evaluation is essential to position

the proposed approach within the current state of the art and to demonstrate its practical advantages, particularly in terms of computational efficiency and suitability for resource-constrained environments consistent with the TinyML paradigm [43].

11.1 Experimental Design

The experimental design of this research is based on the model proposed by Bullinaria (2009), which outlines a neural network architecture with duplicated weights (Dual Weights) as a memory consolidation mechanism to mitigate catastrophic forgetting. The main objective is to extend this approach towards an architecture with multiple duplicated weights, in order to analyse how the number of intermediate memories affects knowledge retention, learning stability, and the associated computational cost.

The experiment will be conducted under a controlled incremental learning scheme, where the neural model will be progressively exposed to different subsets of the Optical Digits dataset (OptDigits). Each subset, referred to as an incremental block, represents a learning stage that introduces new information without explicitly revisiting data from previous stages.

Objective of the experimental design To evaluate the impact of the number of duplicated memories on the performance of an incremental neural network, comparing Bullinaria's baseline version (two memories) with extended architectures of four and six memories.

Experimental variables

- **Independent variable:** number of duplicated weight sets ($n = 2, 4, 6$).
- **Dependent variables:** incremental accuracy (IA), forgetting rate (F), memory retention (RM), and computational cost (CC).

Dataset partitioning strategy The OptDigits dataset contains 10 classes (digits 0–9), with a total of 3,823 training samples and 1,797 test samples distributed uniformly across classes. The dataset will be partitioned into six incremental blocks ($B_1, B_2, \ldots, B_6$) following a **class-incremental learning** scheme [5], where each block introduces a disjoint subset of classes that were not present in previous blocks. Specifically, the partition will be organised as follows:

- B_1: digits $\{0, 1\}$
- B_2: digits $\{2, 3\}$
- B_3: digits $\{4, 5\}$
- B_4: digits $\{6, 7\}$
- B_5: digits $\{8, 9\}$
- B_6: all digits $\{0–9\}$ (full dataset, used as a final global retention evaluation block)

This partitioning strategy is consistent with the class-incremental setting, which is considered one of the most challenging scenarios in continual learning, as the model must recognise all previously seen classes at inference time without access to past training data [5, 30]. Each block will use its corresponding held-out test set for evaluation, and after training on each block, the model will also be evaluated on the test sets of all previous blocks to measure stability and forgetting. For the additional datasets introduced in the extended evaluation (MNIST [17], Fashion-MNIST [45], and Speech Commands [42]), an equivalent class-incremental partitioning strategy will be applied, adapting the number of blocks according to the total number of classes available in each dataset.

General methodology The experiment will be structured in six incremental blocks ($B_1, B_2, \ldots, B_6$), representing different stages of the learning process. Each block will contain disjoint subsets of the training set and will be applied sequentially. After training each block, the model will be evaluated on:

1. Newly learned data (plasticity),
2. Data from previous blocks (stability), and

3. The cumulative dataset (global retention).

The experimental flow is illustrated in Figure 16, where each incremental iteration involves training, evaluation, metric recording, and the transfer of weights to higher-level memories for consolidation.

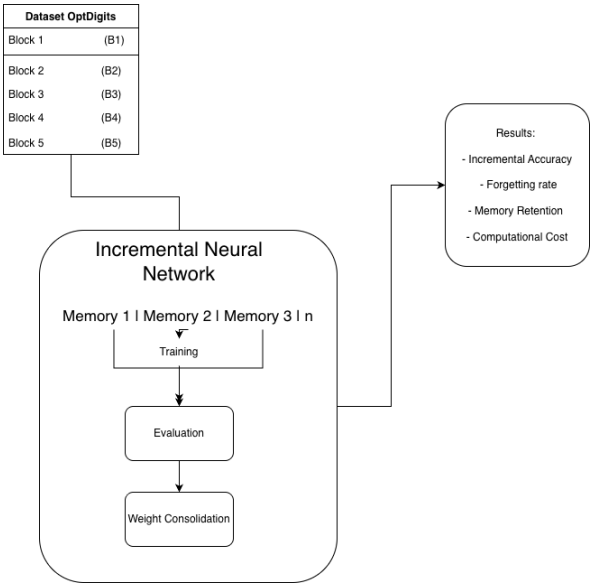

Figure 16: General flow of the proposed incremental experimental design.

The general procedure followed by the proposed model is described in Algorithm 1, which illustrates the flow of training and knowledge consolidation in each incremental block. This process is repeated sequentially until all six learning blocks are completed, while recording the metrics defined in Section 6.2.

Algorithm 1 General procedure of incremental learning

1: **for** each incremental block B_i **do**
2: Train network with B_i
3: Evaluate performance on $B_1, \ldots, B_i$
4: Consolidate weights towards higher memories
5: Record metrics (IA, F, RM, CC)
6: **end for**

The presented flow establishes the operational basis of the experiment, where a controlled training–consolidation cycle is repeated for each incremental block. This scheme aims to verify the validity of the hypothesis concerning the effect of the number of duplicated memories on knowledge retention.

To ensure the reproducibility of the experiment, Table 3 presents the technical configuration used in the implementation of the proposed model. It details the neural network parameters, dataset characteristics, and execution environment employed in the experiments conducted on Apple Silicon architecture.

Table 3: Experimental configuration of the proposed incremental model.

Parameter	Value
Dataset	OptDigits (3,823 training samples / 1,797 test samples, 10 classes)
Base architecture	64–32–10 (fully connected dense layers)
Activation function	Sigmoid
Loss function	Cross Entropy Loss
Optimiser	Adam ($\eta = 0.001$)
Number of incremental blocks	6
Duplicated weight sets	2, 4, and 6
Metrics	IA (incremental accuracy), F (forgetting rate), RM (memory retention), CC (computational cost)
Execution environment	macOS Sonoma, Apple M2 (8 CPU + 10 GPU), 16 GB RAM, PyTorch 2.3.1, Python 3.12

Justification of hyperparameter choices The hyperparameter values presented in Table 3 were selected based on a combination of reproducibility requirements, alignment with prior work, and preliminary considerations regarding the OptDigits dataset characteristics:

- **Base architecture (64–32–10):** The input dimensionality of the OptDigits dataset is 64 features (corresponding to an 8×8 pixel flattened image), which directly determines the input layer size. The hidden layer size of 32 neurons and the output layer of 10 neurons (one per digit class) follow the configuration reported in [3], ensuring reproducibility of the baseline model before extension.
- **Activation function (Sigmoid):** The sigmoid function was chosen to maintain consistency with the original implementation of [3], which is the baseline this work seeks to reproduce and extend. This also facilitates a direct and fair comparison between the original and extended models.
- **Loss function (Cross Entropy Loss):** Cross Entropy Loss is the standard choice for multi-class classification tasks [5], as it directly optimises the probability distribution over the 10 digit classes.
- **Optimiser (Adam, $\eta = 0.001$):** The Adam optimiser was selected due to its well-documented efficiency in training MLPs, particularly its adaptive learning rate adjustment per parameter, which improves convergence stability in incremental settings [14]. The learning rate of 0.001 is the widely adopted default value reported in the original Adam paper and commonly used in continual learning benchmarks [15, 46].
- **Number of incremental blocks (6):** Six blocks were chosen to match the class-incremental partitioning strategy defined in the dataset partitioning section, where each block introduces a pair of new digit classes, covering all 10 classes across five active learning blocks plus one final global evaluation block.
- **Duplicated weight sets (2, 4, 6):** The value of 2 corresponds to the original dual-weight configuration of [3] and serves as the baseline. The values of 4 and 6 were selected to systematically investigate the effect of increasing the number of intermediate memory levels, following a controlled incremental progression that allows identifying whether a clear trend exists between the number of weight sets and knowledge retention performance.

It is worth noting that a more exhaustive hyperparameter search (e.g., using grid search or Bayesian optimisation) is left as future work, as the primary goal of this study is to evaluate the effect of the number of duplicated weight sets under controlled and reproducible conditions consistent with the baseline of [3].

Contrast hypothesis It is expected that the architectures with four and six memories will show greater knowledge retention ($RM \geq 90\%$) and a lower forgetting rate ($F \leq 10\%$) compared to the original dual model, without a significant increase in training time.

Analysis metrics The quantitative metrics used will be those defined in Section 6.2, ensuring methodological consistency. The results will be compared using descriptive statistical analysis and performance curve visualisation per block.

Experimental setup The model will be implemented in Python using TensorFlow and Keras. The execution environment will consist of a workstation equipped with an Intel Core i5 processor, 32 GB of RAM, and macOS, ensuring reproducible conditions. Each architecture will be initialised with the same weights and parameters to maintain experimental fairness.

Design summary The experimental design aims to validate the proposed hypothesis by quantifying the relationship between the number of duplicated memories and the model's stability–plasticity balance. If the hypothesis is confirmed, the results will demonstrate that distributed consolidation based on multiple memories constitutes an efficient and biologically inspired mechanism for incremental learning.

11.2 Expected Results

It is expected that the architectures with four and six duplicated memories will exhibit higher knowledge retention ($RM \geq 90\%$) and a lower forgetting rate ($F \leq 10\%$) compared to Bullinaria's original dual model. Similarly, the computational cost (CC) is anticipated to remain within an acceptable margin, not exceeding a 15% increase relative to the base model, while maintaining an adequate balance between stability and plasticity.

Qualitatively, the extended model is expected to display a smoother transition between the acquisition and consolidation phases of learning, reducing information loss between consecutive blocks. If these trends are confirmed, the results would support the hypothesis that progressive duplication of synaptic weights serves as an efficient internal mechanism for memory consolidation in incremental learning.

Finally, the results obtained will serve as a reference to contrast the effectiveness of the proposed approach with other contemporary methods for mitigating catastrophic forgetting, such as EWC, MAS, or Replay-based Learning.

12 Chapter Organisation

In Chapter 2, the fundamental concepts and functioning of Artificial Neural Networks (ANNs) will be explained, together with the activation functions, which serve as the mathematical mechanisms employed by these networks. The different types of neural networks will be described, including both Multilayer Perceptrons (MLP) and Convolutional Neural Networks (CNN), along with the backpropagation algorithm. The *OptDigits* dataset, which will be used for the training and testing of the algorithms, will also be introduced. Finally, the concept of incremental learning and its associated algorithm will be described.

In Chapter 3, the algorithm proposed by [3] will be implemented. Subsequently, its performance will be verified, and the results obtained using the same dataset mentioned in the original article will be analysed, with the aim of confirming that the algorithm's behaviour corresponds to that reported by the author.

Chapter 4 will explain how the base algorithm was extended to allow the use of more than two layers of duplicated weights, applying the same training and testing data as in the base model.

Subsequently, Chapter 5 will present a comparison between the results obtained from both implementations to determine whether a significant reduction in forgetting rates was achieved. Finally, Chapter 6 will present the conclusions and outline potential directions for future work.

13 Gantt Diagram

The following Gantt chart illustrates the planned timeline of the activities comprising the development of this research. Each phase is organised sequentially and bi-monthly, from the formulation of the problem to the professional examination defence. This representation allows visualising the distribution of tasks, execution periods, and the relationship among the various project stages, ensuring systematic and well-structured monitoring of the overall progress.

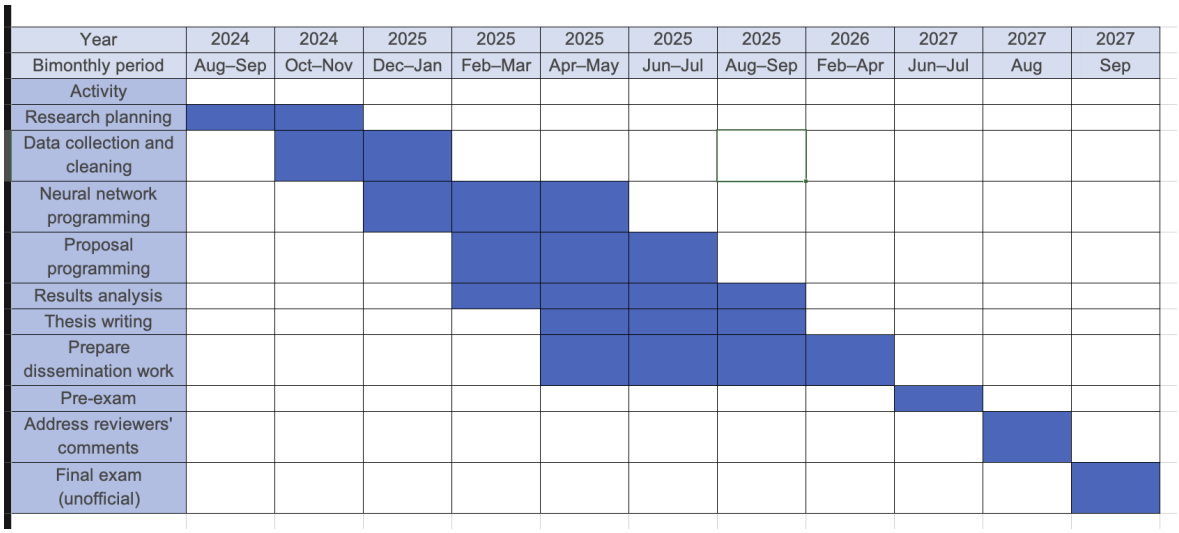

Figure 17: Gantt Chart

References

[1] RR Ade and PR Deshmukh. Methods for incremental learning: a survey. *International Journal of Data Mining & Knowledge Management Process*, 3(4):119, 2013.

[2] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 139–154, 2018.

[3] John A Bullinaria. Evolved dual weight neural architectures to facilitate incremental learning. In *IJCCI*, pages 427–434, 2009.

[4] Francesco Cerasuolo, Alfredo Nascita, Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciuonzo, Antonio Pescapè, and Dario Rossi. Memento: A novel approach for class incremental learning of encrypted traffic. *Computer Networks*, 245:110374, 2024.

[5] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.

[6] Ryan Elwell and Robi Polikar. Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10):1517–1531, 2011.

[7] Douglas H Fisher. Knowledge acquisition via incremental conceptual clustering. *Machine learning*, 2(2):139–172, 1987.

[8] Christophe G. Giraud-Carrier. A note on the utility of incremental learning. *AI Commun.*, 13:215–224, 2000.

[9] Heitor Murilo Gomes, Jean Paul Barddal, Fabrício Enembreck, and Albert Bifet. A survey on ensemble learning for data stream classification. *ACM Computing Surveys*, 2017.

[10] Ian J Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks. *arXiv:1312.6211*, 2013.

[11] Guy Hacohen and Tinne Tuytelaars. Forgetting order of continual learning: What is learned first is forgotten last. `https://arxiv.org/abs/xxxx.xxxxx`, 2023. arXiv preprint.

[12] H. Vega Huerta, A. Cortez Vásquez, Ana María Huayna, Luis Alarcón Loayza, and P. Romero Naupari. Reconocimiento de patrones mediante redes neuronales artificiales. *Revista de investigación de Sistemas e Informática*, 6(2):17–26, 2009.

[13] Rodrigo Jacznik, Mariano Tassara, Iván D'Uva, Guillermo Baldino, Roxana Silvia Giandini, and Leopoldo Nahuel. Integrando modelo de aprendizaje supervisado al análisis del desempeño de alumnos en cursos virtuales sobre plataformas moodle. In *XXII Congreso Argentino de Ciencias de la Computación (CACIC 2016)*, 2016.

[14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[15] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[16] Bartosz Krawczyk, Leandro L. Minku, João Gama, Jerzy Stefanowski, and Michał Woźniak. Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37:132–156, 2017.

[17] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[18] Ai-Jun Li. An improved algorithm for incremental learning learn++. In *2008 International Conference on Wavelet Analysis and Pattern Recognition*, volume 1, pages 310–315. IEEE, 2008.

[19] Derek Li, Zhizhong y Hoiem. Learning without forgetting. *arXiv:1606.09282*, 2016.

[20] Timothy P Lillicrap, Adam Santoro, Luke Marris, Colin J Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

[21] Vittorio Lippi, Cristian Camardella, Alessandro Filippeschi, and Francesco Porcini. Identification of gait phases with neural networks for smooth transparent control of a lower limb exoskeleton, 2021.

[22] Yuan Liu. Incremental learning in deep neural networks, 2015. Unpublished manuscript.

[23] Yuyang Liu, Qiuhe Hong, Linlan Huang, Alexandra Gomez-Villa, Dipam Goswami, Xialei Liu, Joost van de Weijer, and Yonghong Tian. Continual learning for vlms: A survey and taxonomy beyond forgetting. *arXiv preprint arXiv:2508.04227*, 2025.

[24] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6467–6476, 2017.

[25] Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7765–7773, 2018.

[26] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

[27] Luis Miralles Pechuán, Dafne A. Rosso-Pelayo, and Jorge Brieva. Reconocimiento de dígitos escritos a mano mediante métodos de tratamiento de imagen y modelos de clasificación. *Res. Comput. Sci.*, 93:83–94, 2015.

[28] Robi Polikar, Lalita Upda, Satish S. Upda, and Vasant Honavar. Learn++: An incremental learning algorithm for supervised neural networks. *IEEE Transactions on Systems, Man, and Cybernetics*, 31:497–508, 2001.

[29] Darwin Mercado Polo, Luis Pedraza Caballero, and Edinson Martínez Gómez. Comparación de redes neuronales aplicadas a la predicción de series de tiempo. *Prospectiva*, 13(2):88–95, 2015.

[30] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2001–2010, 2017.

[31] V Renganathan. Overview of artificial neural network models in the biomedical domain. *Bratislava Medical Journal/Bratislavské Lekárske Listy*, 120(7), 2019.

[32] Raúl Rojas. *The Backpropagation Algorithm*, pages 149–182. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996.

[33] David Rolnick, Arun Ahuja, Jonathan Schwartz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 350–360, 2019.

[34] Paloma Royo. Qué son las redes neuronales y cuál es su aplicación en el marketing, 2021.

[35] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[36] Luis A Cruz Salazar, David J Muñoz Aldana, and Juan A Contreras Montes. Implementación de redes neuronales y lógica difusa para la clasificación de patrones obtenidos por un sónar. In *2013 II International Congress of Engineering Mechatronics and Automation (CIIMA)*, pages 1–6. IEEE, 2013.

[37] Krisanu Sarkar. Adaptive variance-penalized continual learning with fisher regularization. *arXiv preprint arXiv:2508.16632*, 2025.

[38] Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*, 2024.

[39] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2990–2999, 2017.

[40] Gido M. van de Ven and Andreas S. Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.

[41] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive study of catastrophic forgetting in language models. *arXiv preprint arXiv:2402.07786*, 2024.

[42] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition, 2018. arXiv preprint arXiv:1804.03209.

[43] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[44] Xing Wei, Shaofan Liu, Yaoci Xiang, Zhangling Duan, Chong Zhao, and Yang Lu. Incremental learning based multi-domain adaptation for object detection. *Knowledge-Based Systems*, 210:106420, 2020.

[45] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

[46] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3987–3995. PMLR, 2017.

[47] Beibei Zhao, Tairan Wu, Fang Fang, Lin Wang, Wenzhang Ren, Xu Yang, Zhangjing Ruan, and Xuejin Kou. Prediction method of 5g high-load cellular based on bp neural network. In *2022 8th International Conference on Mechatronics and Robotics Engineering (ICMRE)*, pages 148–151. IEEE, 2022.

A Phases of the Backpropagation Process

A.1 Forward Propagation

In this phase, the input values are introduced into the network, and the output is computed. There is no weight adjustment at this stage, as only the activation value is calculated.

This process is achieved through matrix multiplication, where each neuron receives the outputs of the neurons from the previous layer, multiplied by their respective weights, as described by the following equation:

$$z_j = \sum_i (w_{ji} \cdot x_i) + b_j$$

Where:

- w_{ji} = weight of the connection between neuron i in the previous layer and neuron j in the current layer.
- x_i = output of neuron i in the previous layer.
- b_j = bias of neuron j.

A.2 Backpropagation

In this phase, the network weights are adjusted by propagating the error backwards—from the output layer to the hidden layers—using the gradient descent algorithm. To achieve this, the error must first be calculated by defining the loss function as:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Once the loss is computed, the backward propagation is carried out using the chain rule of differential calculus. This process is divided into two parts: the calculation for the output layer and that for the hidden layers.

- **Output Layer j:**

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j}$$

  Where:

  - $\frac{\partial L}{\partial \hat{y}_j} = (\hat{y}_j - y_j)$ represents the derivative of the loss function.
  - $\frac{\partial \hat{y}_j}{\partial z_j}$ depends on the activation function. For example, if the sigmoid function is used:

$$\sigma'(z_j) = \sigma(z_j)(1 - \sigma(z_j))$$

- **Hidden Layers:**

$$\frac{\partial L}{\partial z_j} = \sum_k \left(\frac{\partial L}{\partial z_k} \cdot w_{kj} \cdot \sigma'(z_j) \right)$$

  Here, w_{kj} represents the weights from the next layer.

Finally, the weights (w_{ji}) and biases (b_j) are updated using the gradient and a learning rate (η):

$$w_{ji} \leftarrow w_{ji} - \eta \cdot \frac{\partial L}{\partial w_{ji}}$$

$$b_j \leftarrow b_j - \eta \cdot \frac{\partial L}{\partial b_j}$$

Where:

$$\frac{\partial L}{\partial w_{ji}} = \frac{\partial L}{\partial z_j} \cdot x_i \quad \text{and} \quad \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial z_j}$$